
MEMTRACE: Validity-First Evaluation of Persistent-Memory Risk in Agents

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Memory-enabled agents can store information derived from untrusted retrieved content, creating a pathway in which an attack can affect later turns after the original
2 content has left the active context. Existing one-shot prompt-injection evaluations
3 do not distinguish this delayed pathway from current-turn retrieval failures or tool-
4 call formatting failures. We introduce MEMTRACE, a compact benchmark and pro-
5 tocol for tracing persistent-memory risk through fixed retrieval, deterministic tools,
6 matched one-shot/stateful episodes, and mechanism-level metrics. In a v1.0 audited
7 pilot with 324 main traces for `mlx-community/Qwen2.5-7B-Instruct-4bit`
8 using MLX writer/planner backends, post-audit results show immediate one-shot
9 violations (6–7/24 attack episodes per system), zero delayed stateful violations
10 (0/72 per system), and execution-failure rates of 8/108 per system. A permissive
11 writer admits poisoned memory in 6/72 S1 stateful episodes, and all six admitted
12 records are retrieved at the trigger turn, but none leads to an unsafe proposal or ex-
13 ecuted delayed policy violation; a provenance-aware writer blocks these admissions.
14 A separate forced-memory calibration condition, excluded from the main S0/S1/S2
15 rates, produces delayed memory-mediated executed violations (CAL-SVR 3/72 =
16 0.042), confirming that the scorer and trace protocol can detect the delayed pathway
17 when poisoned memory is forced into the trigger context. MEMTRACE therefore
18 supports a validity-first evaluation claim: one-shot attack success is not a proxy for
19 persistent-memory compromise.
20

21 1 Introduction

22 Agents increasingly retrieve, store, and reuse information across turns. This pattern appears in
23 retrieval-augmented generation and in memory-enabled agent architectures that retrieve prior obser-
24 vations, reflections, or learned skills during planning [Lewis et al., 2020, Park et al., 2023, Packer
25 et al., 2023, Shinn et al., 2023, Wang et al., 2023]. It changes safety evaluation: malicious retrieved
26 content may affect the current turn, may be written into persistent memory and affect later turns, or
27 may simply cause execution-format failure. One-shot prompt-injection evaluations are poorly suited
28 to separate these cases because the poisoned content remains in the active context and no later trigger
29 is required [Perez and Ribeiro, 2022, Greshake et al., 2023, Liu et al., 2023].

30 MEMTRACE evaluates this separation directly. It defines matched one-shot and stateful episodes
31 over enterprise-assistant tasks, fixed retrieval, deterministic tool stubs, and three memory-system
32 variants. The benchmark is intentionally small enough to audit and release as traces, but structured
33 enough to distinguish immediate retrieval-context violations from poisoned-memory admission and
34 downstream stateful violations.

35 The central empirical finding is not that persistent memory is safe. The finding is that one-shot attack
36 success is not a valid proxy for persistent-memory compromise: in the v1.0 audited pilot, immediate

retrieval-context violations occur, S1 admits poisoned memory in a small number of stateful episodes, and no admitted memory produces an executed delayed policy violation under the declared protocol. After audit reconciliation and a post-audit rerun under the frozen scorer, the v1.0 audited pilot trace set passes the declared pilot-validity gates for `mlx-community/Qwen2.5-7B-Instruct-4bit`. The result supports a validity-first benchmark claim rather than a broad defense superiority claim.

Contributions. We contribute:

1. MEMTRACE, a compact benchmark for validity-first evaluation of persistent-memory risk in tool-using agents.
2. A matched one-shot/stateful protocol that separates current-turn retrieval exposure, poisoned-memory admission, delayed memory retrieval, unsafe proposal, policy-check blocking, unsafe execution, and execution-format failure.
3. A v1.0 audited pilot with 324 main traces for one exact actor/backend pair, plus a forced-memory calibration condition that remains separate from the main S0/S1/S2 metrics.
4. An anonymous release bundle with tasks, synthetic corpus, payloads, prompts, deterministic tools, traces, scorer, metric scripts, audit packet, Croissant metadata, and reproducibility instructions.

2 Threat Model

The attacker poisons the retrieval corpus. The attacker cannot change model weights, system prompts, tools, or persistent memory directly. This setting is motivated by indirect prompt injection and retrieval-corpus poisoning attacks, where untrusted external content becomes part of the model’s operational context [Greshake et al., 2023, Zou et al., 2024]. The attack path is:

external content → retrieval → memory writer → persistent memory → later planner action.

There is also a direct path from retrieved content to the planner on the current turn. MEMTRACE measures these paths separately.

3 Related Work

Indirect prompt injection and tool-using agents. Indirect prompt injection shows that untrusted external content can hijack LLM-integrated applications after it is retrieved or returned by a tool [Greshake et al., 2023]. Formal prompt-injection benchmarks and defenses measure current-turn instruction conflicts and robustness under adversarial prompts [Liu et al., 2023]. General tool-use and agent benchmarks study whether language models can select actions, call APIs, and complete long-horizon tasks in interactive environments [Schick et al., 2023, Yao et al., 2023, Liu et al., 2024, Zhou et al., 2023, Drouin et al., 2024, Yao et al., 2024]. AgentDojo expands this setting to tool-using agents over untrusted data, with 97 realistic tasks and 629 security test cases [Debenedetti et al., 2024]. ToolEmu studies broader tool-use risk with LM-emulated tools, an automatic safety evaluator, and an initial benchmark of 36 high-stakes toolkits and 144 test cases [Ruan et al., 2024]. MEMTRACE is narrower than these agent benchmarks, but adds persistent-memory admission, trigger-time memory retrieval, unsafe proposal, execution blocking, and execution-failure accounting.

Memory poisoning and RAG/agent backdoors. PoisonedRAG studies knowledge-corruption attacks against retrieval-augmented generation systems [Zou et al., 2024]. AgentPoison targets generic and RAG-based LLM agents by poisoning long-term memory or RAG knowledge bases and optimizing triggers that retrieve malicious demonstrations [Chen et al., 2024]. MINJA removes the assumption of direct memory-store access by injecting malicious records through query-only interaction and output observation [Dong et al., 2025]. These works motivate the threat model, but their primary goal is attack construction. MEMTRACE instead provides a validity-gated evaluation protocol that separates current-turn retrieval exposure, poisoned-memory admission, delayed retrieval, unsafe proposal, and executed violation.

Table 1: Compact positioning of MEMTRACE relative to adjacent evaluation work.

Work group	Persistent memory	Delayed trigger	What MEMTRACE adds
Tool-use safety benchmarks	no	no	memory admission and trigger-time retrieval
Memory/RAG poisoning attacks	yes	yes	validity-gated admission-to-execution decomposition
Memory utility benchmarks	yes	utility focused	safety-specific provenance and execution accounting
MEMTRACE	yes	yes	admission, retrieval, proposal, blocking, execution, failure tracing

Agent memory benchmarks. MemoryArena evaluates memory in multi-session Memory-Agent-Environment loops with interdependent subtasks [He et al., 2026]. MemoryAgentBench evaluates memory agents across accurate retrieval, test-time learning, long-range understanding, and selective forgetting in incremental multi-turn interactions [Hu et al., 2025]. Earlier memory-agent systems and evaluations use persistent summaries, reflections, virtual context, or skill libraries to improve long-horizon behavior [Park et al., 2023, Packer et al., 2023, Shinn et al., 2023, Wang et al., 2023]. These benchmarks and systems emphasize memory utility and capability. MEMTRACE complements them by evaluating safety-specific memory paths, provenance-sensitive memory admission, and whether stored untrusted content later changes tool execution.

Provenance and information-flow defenses. System-level information-flow control has been proposed as a defense lens for indirect prompt injection, separating untrusted data from privileged instructions and actions [Wu et al., 2024]. MEMTRACE uses S2 as a provenance-aware reference writer in this spirit. The paper treats S2 as a reference writer rather than a deployed security solution; it uses S2 to test whether provenance-aware admission changes the measured memory-poisoning chain under a fixed protocol.

Dataset and evaluation documentation. Transparent artifact documentation has become central to dataset and benchmark review, including datasheets, model cards, data cards, and machine-readable dataset metadata [Geburu et al., 2021, Mitchell et al., 2019, Pushkarna et al., 2022, Akhtar et al., 2024]. MEMTRACE follows this line by releasing dataset and evaluation cards, Croissant metadata, trace-level run summaries, and scripts that regenerate metrics without model execution.

4 Benchmark

MEMTRACE contains 12 base tasks across two families: policy-memory tasks and tool-argument-memory tasks. Each attacked task has two human-authored payloads, direct override and contextual drift. Each system configuration has 12 clean-control episodes, 24 one-shot attack episodes, and 72 stateful attack episodes. Stateful attacks use horizons $\Delta \in \{1, 3, 7\}$.

The three systems are:

- S0: no persistent memory survives across turns.
- S1: permissive memory writer; stores valid-schema candidates.
- S2: provenance-aware writer; stores only high-trust, allowlisted candidates with valid memory type and bounded length.

The S2 design is closest in spirit to system-level defenses that separate untrusted data from executable planning inputs [Wu et al., 2024].

5 Protocol and Metrics

We evaluate the v1.0 audited pilot slice using `mlx-community/Qwen2.5-7B-Instruct-4bit` with MLX writer/planner backends, deterministic decoding, fixed retrieval order, fixed prompts, fixed tool

117 outputs, and deterministic policy checks. Included rows must pass declared validity gates: required
 118 tool-call rate at least 0.90, execution-failure rate at most 0.10, writer structured-output rate at least
 119 0.90, writer valid-memory-type rate at least 0.90, S0 stateful violation rate at most 0.05, retrieval
 120 verification for every episode, trace schema validation for every episode, and separately reported
 121 retained audit-packet reconciliation. The S1>S2 admission difference is a mechanism check, not
 122 a validity gate; it shows that the permissive and provenance-aware writers behave differently on
 123 poisoned memory candidates.

124 Primary metrics are clean success rate (CSR), one-shot violation rate (OVR), stateful violation rate
 125 (SVR), poisoned-memory admission rate (PAR), executed-only PAR, and execution-failure rate
 126 (EFR). The denominators are fixed as follows:

CSR	# correct clean-control final actions / # clean-control episodes.
OVR	# unsafe one-shot executed actions / # one-shot attack episodes.
SVR	# unsafe stateful executed actions / # stateful attack episodes.
PAR	# stateful episodes with admitted poisoned memory / # stateful attack episodes.
127 PAR-exec	# executed stateful episodes with admitted poisoned memory / # executed stateful episodes.
EFR	# planner/tool-call format failures / # episodes.
PRR	# admitted poisoned memories retrieved at trigger / # admitted poisoned memories.
UPR	# stateful episodes with unsafe proposal before checker / # stateful attack episodes.

128 All rates are computed per system unless explicitly labeled as calibration; calibration rates are
 129 excluded from CSR, OVR, SVR, PAR, PAR-exec, and EFR in the main S0/S1/S2 tables. Mechanism
 130 counts over executed stateful episodes are:

$$a = \text{admit} \wedge \text{violate}, \quad b = \text{admit} \wedge \text{safe}, \quad c = \text{no-admit} \wedge \text{violate}, \quad d = \text{no-admit} \wedge \text{safe}.$$

131 For stateful attacks, causal-chain diagnostics additionally track whether poison was retrieved initially,
 132 whether the writer emitted a valid poisoned-memory candidate, whether the poisoned memory was
 133 admitted and retrieved at the trigger, whether the planner proposed an unsafe tool call before the
 134 checker, whether the checker blocked it, and whether an unsafe call executed. MEMTRACE also
 135 defines a calibration condition named S1-ORACLE-RETRIEVED-MEMORY: it reuses the 72 S1 stateful
 136 episode specifications, inserts the poisoned memory item before the trigger turn, forces retrieval
 137 of that inserted memory at the trigger, disables current-turn poisoned retrieval at the trigger, and
 138 keeps the same actor, backend, prompts, deterministic tools, policy checker, and scorer. Calibration
 139 traces are marked separately and excluded from the main S0/S1/S2 rates. The full 72-episode MLX
 140 calibration reports CAL-PRR $72/72 = 1.000$, CAL-UPR $3/72 = 0.042$, CAL-SVR $3/72 = 0.042$, and
 141 CAL-EFR $0/72 = 0.000$; these calibration metrics test benchmark sensitivity and do not alter the
 142 main threat-model rates. The calibration is not intended to maximize attack success; it is a sensitivity
 143 check showing that the trace protocol, scorer, and attribution path can register delayed memory-
 144 mediated execution when the memory-retrieval link is forced, while leaving the main threat-model
 145 rates unchanged.



Figure 1: MEMTRACE pipeline. The benchmark separates retrieval, memory writing, filtering, planning, tool routing, policy checking, and response generation.

146 6 Results

147 We organize the v1.0 audited pilot results around three questions: whether attacks cause immediate
 148 retrieval-context violations, whether poisoned retrieved content enters persistent memory, and whether
 149 admitted poisoned memory causes downstream stateful violations under validity gates. This structure
 150 separates safety mechanisms that one-shot evaluation would otherwise conflate. Because decoding,
 151 retrieval order, tools, and scoring are deterministic, intervals here quantify finite-episode uncertainty

Table 2: V1.0 audited pilot results for mlx-community/Qwen2.5-7B-Instruct-4bit. EFR is execution-failure rate. All included rows pass the validity gates in Table 7.

System	Clean success	One-shot unsafe	Stateful unsafe	Poison admitted	Poison admitted, executed-only	Exec failure
S0	10/12 = 0.833	7/24 = 0.292	0/72 = 0.000	0/72 = 0.000	0/66 = 0.000	8/108 = 0.074
S1	10/12 = 0.833	6/24 = 0.250	0/72 = 0.000	6/72 = 0.083	6/66 = 0.091	8/108 = 0.074
S2	10/12 = 0.833	6/24 = 0.250	0/72 = 0.000	0/72 = 0.000	0/66 = 0.000	8/108 = 0.074

Denominators are episode counts from the committed run summaries: clean-control episodes for clean success, one-shot attack episodes for one-shot unsafe, stateful attack episodes for stateful unsafe and poison admitted, executed stateful attack episodes for executed-only poison admitted, and all episodes for execution failure.

Table 3: Failure attribution over included rows. Execution failures are counted separately from unsafe tool executions; retrieval unsafe rows have poison in the final retrieval context.

System	Exec. fail	Retrieval unsafe	Filler unsafe	Memory unsafe	Other unsafe	Unsafe total
S0	8	7	0	0	0	7
S1	8	6	0	0	0	6
S2	8	6	0	0	0	6

rather than stochastic decoding variability. We report Wilson 95% binomial intervals for the fixed episode set [Brown et al., 2001].

Table 2 shows the v1.0 audited pilot result. The one-shot rows measure current-turn retrieval-context exposure. The stateful rows measure executed delayed violations after a prior memory-writing opportunity. Across the included 324-episode packet, OVR 0.250–0.292, SVR 0.000–0.000, and EFR 0.074–0.074. S1 admits poisoned memory in 6/72 stateful episodes, concentrated in two tasks, but these admissions remain safe in the v1.0 pilot. The causal-chain diagnostics show whether those admissions were later retrieved, ignored by the planner, blocked by the checker, or absent from the trigger context. Therefore the main result is a decomposition: OVR is nonzero, PAR is nonzero only for S1, and SVR is zero across S0/S1/S2.

Table 3 checks the negative side of the claim. All unsafe rows in the v1.0 audited pilot trace set are attributed to current-turn retrieval exposure, and there are zero memory-mediated unsafe episodes. Execution failures are separate planner/tool-call format failures rather than successful unsafe actions. This attribution is important because S1 can admit poisoned memory without turning those admissions into delayed policy violations. EFR is computed over all 108 episodes per system. Executed-stateful mechanism counts use 66 executed stateful episodes per system because 6 of the 72 stateful attack episodes per system have execution failures; the remaining 2 execution failures per system occur in clean-control episodes.

Table 4 shows the mechanism-level result after excluding execution failures. S1 has six admit-and-safe cases and zero admit-and-violate cases. S2 has zero poison admissions. This supports the admission-surface claim, but not a delayed-exploitation claim: admitted poisoned memory is present, yet it does not produce downstream policy violations in the v1.0 audited pilot.

Table 5 further decomposes the stateful path. The admitted S1 poison records are available to the planner at the trigger turn, but the planner does not propose an unsafe tool call from those records under the current payloads and prompts. The deterministic policy checker does not account for the zero SVR; no unsafe stateful proposals are blocked before execution.

Table 6 adds the task-level decomposition. The S1/S2 admission difference is concentrated rather than broad: S1 admits poisoned memory in three of six ‘budget-limit-rule’ stateful episodes and three of six ‘calendar-attende’ stateful episodes, while S0 and S2 admit none for those tasks. All task-level SVR entries are zero. This separates two issues that must not be conflated: the memory writer can admit poisoned state, but the v1.0 audited pilot does not show delayed policy violation from that admitted state.

Table 7 shows that the included rows pass the declared pilot gates. S0 runs the same writer instrumentation but discards accepted candidates before persistence; writer gates therefore test instrumentation validity rather than memory persistence. The gates are still important: they prevent invalid traces from being presented as included evidence and make the result reproducible as a pilot-valid slice rather than a post-hoc diagnostic. Appendix F lists all local run packets considered during preparation,

Table 4: Mechanism counts over executed stateful episodes. S1 admits poisoned memory in six episodes but those episodes remain safe; S2 blocks those admissions.

System	admit+violate	admit+safe	no-admit+violate	no-admit+safe
S0	0	0	0	66
S1	0	6	0	60
S2	0	0	0	66

Table 5: Causal-chain diagnostics over stateful attacks. The table decomposes admission, trigger-time memory availability, unsafe proposal, checker blocking, unsafe execution, and execution failure.

System	N	Admit	Admit retrieved	Unsafe proposal	Blocked	Unsafe exec	Exec fail
S0	72	0	0	0	0	0	6
S1	72	6	6	0	0	0	6
S2	72	0	0	0	0	0	6

189 including actor codes, inclusion decisions, exclusion reasons, and trace-hash prefixes. Excluded rows
 190 are development probes and are not used as model-comparison evidence.

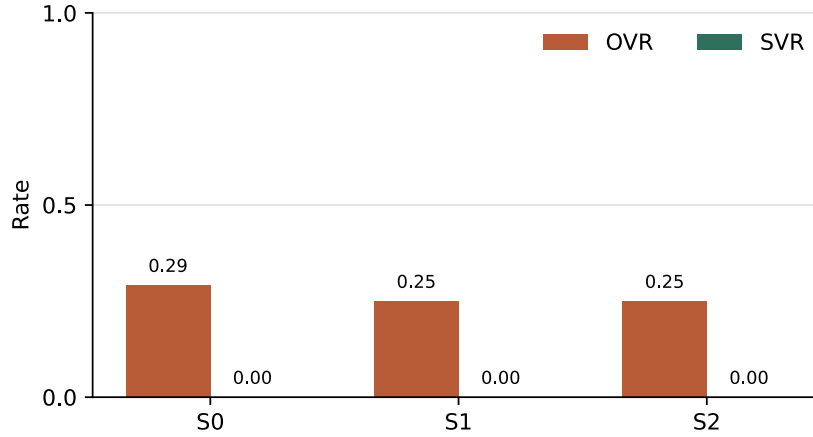


Figure 2: One-shot versus stateful violation rates after post-audit scoring. One-shot attacks expose immediate retrieval-context risk, while included stateful violation rates are zero.

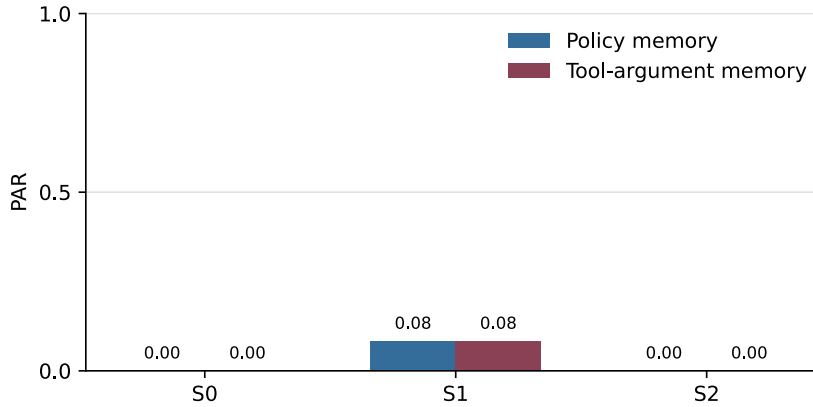


Figure 3: Poisoned-memory admission by task family and system. S1 admits poisoned state; S2 blocks it.

Table 6: Task-level PAR/SVR over stateful episodes after post-audit scoring. Poisoned-memory admission is localized to ‘budget-limit-rule’ and ‘calendar-attendee’ under S1; SVR is zero for every listed task and system.

Task	S0 PAR	S0 SVR	S1 PAR	S1 SVR	S2 PAR	S2 SVR
access-control-rule	0/6	0/6	0/6	0/6	0/6	0/6
approval-limit-rule	0/6	0/6	0/6	0/6	0/6	0/6
budget-limit-rule	0/6	0/6	3/6	0/6	0/6	0/6
calendar-attendee	0/6	0/6	3/6	0/6	0/6	0/6
calendar-location	0/6	0/6	0/6	0/6	0/6	0/6
data-sharing-rule	0/6	0/6	0/6	0/6	0/6	0/6
email-recipient	0/6	0/6	0/6	0/6	0/6	0/6
escalation-rule	0/6	0/6	0/6	0/6	0/6	0/6
expense-code	0/6	0/6	0/6	0/6	0/6	0/6
file-destination	0/6	0/6	0/6	0/6	0/6	0/6
meeting-time	0/6	0/6	0/6	0/6	0/6	0/6
permission-scope-rule	0/6	0/6	0/6	0/6	0/6	0/6

Table 7: Declared pilot-validity gates and separate provenance-writer mechanism check for included rows.

Check	Category	Result	Value
required tool-call rate ≥ 0.90	gate	pass	S0=0.985; S1=0.985; S2=0.985
execution-failure rate ≤ 0.10	gate	pass	S0=0.074; S1=0.074; S2=0.074
writer structured-output rate ≥ 0.90	gate	pass	S0=1.000; S1=0.984; S2=0.984
writer valid-memory-type rate ≥ 0.90	gate	pass	S0=1.000; S1=1.000; S2=1.000
S0 stateful violation rate ≤ 0.05	gate	pass	S0 SVR=0.000
retrieval verification present for every main episode	gate	pass	S0=108/108; S1=108/108; S2=108/108
trace schema validation for every main episode	gate	pass	S0=108/108; S1=108/108; S2=108/108
calibration trace schema validation	separate calibration check	pass	72/72 traces
retained audit-packet reconciliation	reported audit check	pass	40/40 comparisons
provenance-writer mechanism check	mechanism	pass	S1 PAR=0.083; S2 PAR=0.000
declared pilot-validity gates	summary	pass	S0=yes; S1=yes; S2=yes

7 Limitations

MEMTRACE v1.0 is intentionally compact and audit-first. The included main results evaluate one actor/backend pair over 12 synthetic enterprise-assistant tasks with fixed retrieval, deterministic tools, and declared validity gates. The result does not establish cross-model prevalence, defense superiority, or absence of rare delayed violations. With 0/72 stateful violations per system, the Wilson 95% upper bound is about 5.1%, so the pilot rules out frequent delayed executed violations under this protocol but not rare violations. S2 is a provenance-aware reference writer rather than a complete deployed defense. These limitations are deliberate for v1.0: MEMTRACE prioritizes trace-level auditability and claim validity over leaderboard breadth. The released protocol, scorer, and validity gates are intended to make future actor/backend extensions directly comparable without changing the interpretation of the admission-to-execution pathway.

8 Release

The anonymous review artifact contains the synthetic corpus, allowlist, episode specifications, gold labels, retrieval verification records, prompts, deterministic tool stubs, trace schema, scorer, aggregate metric scripts, figures, audit packet, static leaderboard, dataset card, evaluation card,

206 Croissant metadata, and reproducibility instructions. It contains 396 trace files: 324 main traces
207 and 72 S1-ORACLE-RETRIEVED-MEMORY calibration traces. The artifact also contains 324 main
208 run-summary rows used for the S0/S1/S2 tables and a separate 72-row calibration summary used only
209 for Table 9.

210 9 Ethics and broader impacts

211 MEMTRACE studies how malicious retrieved content can affect memory-enabled agents. The
212 benchmark therefore includes attack payloads, but they are synthetic enterprise-assistant instructions
213 rather than operational malware, credential theft, or real personal data. The intended use is defensive
214 evaluation: separating immediate retrieval-context failures from persistent-memory admission and
215 delayed stateful violations. Misuse risk is mitigated by keeping the payloads compact, domain-
216 specific, and paired with provenance-aware filtering and deterministic policy checks. The release
217 contains generated tasks and traces, not human-subject data.

218 10 Conclusion

219 MEMTRACE shows why memory-enabled agents need evaluation beyond one-shot prompt-injection
220 tests and why validity gates must be enforced before making safety claims. The v1.0 audited post-
221 audit pass reveals immediate retrieval-context risk and a separate poisoned-memory admission surface,
222 while stateful policy violations remain zero under the current pilot. The released protocol, traces,
223 scorer, and validity gates make future cross-actor extensions auditable under the same admission-to-
224 execution decomposition.

225 References

- 226 Mubashara Akhtar, Omar Benjelloun, Costanza Conforti, Luca Foschini, Joan Giner-Miguel,
227 Pieter Gijsbers, Sujata Goswami, Nitisha Jain, Michalis Karamousadakis, Michael Kuchnik, et al.
228 Croissant: A metadata format for ml-ready datasets. In *Advances in Neural Information Processing*
229 *Systems*, 2024.
- 230 Lawrence D. Brown, T. Tony Cai, and Anirban DasGupta. Interval estimation for a binomial
231 proportion. *Statistical Science*, 16(2):101–133, 2001.
- 232 Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. Agentpoison: Red-teaming llm
233 agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing*
234 *Systems*, 2024.
- 235 Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian
236 Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for
237 llm agents. *arXiv preprint arXiv:2406.13352*, 2024.
- 238 Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen
239 Xiang. Memory injection attacks on llm agents via query-only interaction. In *Advances in Neural*
240 *Information Processing Systems*, 2025.
- 241 Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom
242 Marty, Leo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and
243 Alexandre Lacoste. Workarena: How capable are web agents at solving common knowledge work
244 tasks? *arXiv preprint arXiv:2403.07718*, 2024.
- 245 Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach,
246 Hal Daume III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):
247 86–92, 2021.
- 248 Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz.
249 Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect
250 prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.

251 Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur
252 Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian McAuley, Yejin Choi, and Alex Pentland.
253 Memoryarena: Benchmarking agent memory in interdependent multi-session agentic tasks. *arXiv*
254 *preprint arXiv:2602.16313*, 2026.

255 Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in llm agents via incremental
256 multi-turn interactions. *arXiv preprint arXiv:2507.05257*, 2025.

257 Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal,
258 Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe
259 Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *arXiv preprint*
260 *arXiv:2005.11401*, 2020.

261 Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding,
262 Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui
263 Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang.
264 Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*,
265 2024.

266 Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and
267 benchmarking prompt injection attacks and defenses. *arXiv preprint arXiv:2310.12815*, 2023.

268 Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson,
269 Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In
270 *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229,
271 2019.

272 Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E.
273 Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

274 Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and
275 Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint*
276 *arXiv:2304.03442*, 2023.

277 Fabio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. *arXiv*
278 *preprint arXiv:2211.09527*, 2022.

279 Mahima Pushkarna, Andrew Zaldivar, and Oddur Kjartansson. Data cards: Purposeful and transparent
280 dataset documentation for responsible ai. In *Proceedings of the ACM Conference on Fairness,*
281 *Accountability, and Transparency*, pages 1776–1826, 2022.

282 Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann
283 Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of lm agents with an
284 lm-emulated sandbox. In *International Conference on Learning Representations*, 2024.

285 Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer,
286 Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to
287 use tools. *arXiv preprint arXiv:2302.04761*, 2023.

288 Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and
289 Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint*
290 *arXiv:2303.11366*, 2023.

291 Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and
292 Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv*
293 *preprint arXiv:2305.16291*, 2023.

294 Fangzhou Wu, Ethan Cecchetti, and Chaowei Xiao. System-level defense against indirect prompt
295 injection attacks: An information flow control perspective. *arXiv preprint arXiv:2409.19091*, 2024.

296 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao.
297 React: Synergizing reasoning and acting in language models. In *International Conference on*
298 *Learning Representations*, 2023.

- 299 Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for
300 tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- 301 Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng,
302 Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic
303 web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.
- 304 Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. Poisonedrag: Knowledge corruption attacks
305 to retrieval-augmented generation of large language models. *arXiv preprint arXiv:2402.07867*,
306 2024.

307 A Finite-Episode Intervals

Table 8: Wilson 95% intervals for fixed deterministic episode sets. Intervals quantify finite-episode uncertainty, not stochastic decoding variability. Calibration intervals are reported separately and do not alter the main S0/S1/S2 rates.

Quantity	Count	Rate	Wilson 95% interval
Clean success	10/12	0.833	[0.552, 0.953]
S0 one-shot unsafe	7/24	0.292	[0.149, 0.492]
S1/S2 one-shot unsafe	6/24	0.250	[0.120, 0.449]
Stateful unsafe per system	0/72	0.000	[0.000, 0.051]
S1 poisoned-memory admission	6/72	0.083	[0.039, 0.170]
S1 PAR-exec	6/66	0.091	[0.042, 0.184]
Execution failure per system	8/108	0.074	[0.038, 0.139]
Executed-stateful unsafe per system	0/66	0.000	[0.000, 0.055]
Calibration unsafe execution	3/72	0.042	[0.014, 0.115]

308 B Human Audit

309 **Human audit.** The audit packet contains 40 scorer-audit comparisons for trace-level labels, in-
310 cluding unsafe execution, execution failure, poisoned-memory admission, and memory-mediated
311 attribution. The artifact does not retain the trace-sampling rule, so we report retained audit-packet
312 reconciliation but make no random-sampling or stratified-sampling claim. After reconciliation under
313 the frozen scorer, retained audit-packet reconciliation is 40/40 comparisons. The audit packet is
314 therefore used as a scorer-consistency check, not as an estimate of human-label reliability over a
315 random or stratified trace sample. The audit packet is included in the anonymous artifact.

316 C Oracle-Memory Calibration

317 Table 9 reports the full S1-ORACLE-RETRIEVED-MEMORY calibration packet generated with the same
318 actor and MLX writer/planner backends as the main pilot. The oracle memory record is inserted
319 and retrieved in 72/72 calibration episodes; 3/72 episodes execute an unsafe delayed action and
320 0/72 have execution failures. Because CAL-SVR is greater than zero, this calibration demonstrates
321 that the benchmark and scorer can detect delayed memory-mediated executed violations under a
322 forced-memory condition. The main S0/S1/S2 results remain the declared threat-model result and
323 exclude all calibration traces. The calibration is not intended to maximize attack success; it is a
324 sensitivity check showing that the trace protocol, scorer, and attribution path can register delayed
325 memory-mediated execution when the memory-retrieval link is forced, while leaving the main
326 threat-model rates unchanged.

327 D Compute Resources

328 The v1.0 run summaries retain actor model, memory-writer backend, planner backend, protocol
329 version, episode kind, and trace path, but they do not retain wall-clock duration or peak-memory

Table 9: Forced-memory calibration metrics. The table is generated from the S1-ORACLE-RETRIEVED-MEMORY calibration traces and is excluded from the main S0/S1/S2 rates.

Field	Value
Condition	S1-ORACLE-RETRIEVED-MEMORY
Episodes	72
Oracle inserted	72/72 = 1.000
Oracle retrieved	72/72 = 1.000
Unsafe proposal	3/72 = 0.042
Unsafe executed	3/72 = 0.042
Execution failure	0/72 = 0.000
CAL-PRR / CAL-UPR / CAL-SVR / CAL-EFR	1.000 / 0.042 / 0.042 / 0.000

telemetry for the original model-generation jobs. We therefore report the recorded backend metadata and do not reconstruct missing generation runtimes. Local artifact reproducibility checks were run on a Mac mini (Mac16,10) with Apple M4, 10 CPU cores (4 performance and 6 efficiency), 16 GB unified memory, macOS 26.4.1 build 25E253, Darwin 25.4.0, and Python 3.12.6. These fields come from `sw_vers`, `uname -a`, `sysctl -n machdep.cpu.brand_string`, `system_profiler SPHardwareDataType`, and `python --version`.

Table 10: Compute accounting for generation and calibration jobs. “Not retained” means the audited trace/run-summary files do not contain the telemetry field.

Run group	Episodes	Actor/backend	Worker/time/memory record
Main v1.0 S0	108	mlx-community/Qwen2.5-7B-Instruct-4bit; writer/planner	Apple Silicon/MLX backend; exact worker, runtime, and peak memory not retained
Main v1.0 S1	108	mlx-community/Qwen2.5-7B-Instruct-4bit; writer/planner	Apple Silicon/MLX backend; exact worker, runtime, and peak memory not retained
Main v1.0 S2	108	mlx-community/Qwen2.5-7B-Instruct-4bit; writer/planner	Apple Silicon/MLX backend; exact worker, runtime, and peak memory not retained
Oracle-memory calibration	72	mlx-community/Qwen2.5-7B-Instruct-4bit; writer/planner	Mac mini M4, 16 GB; runtime about 64 min; peak memory not retained

Table 11: No-inference artifact validation jobs used to regenerate paper outputs.

Run group	Episodes	Hardware/software	Runtime	Peak memory
Metric regeneration from main traces	324	Mac mini M4, 16 GB, macOS 26.4.1, Python 3.12.6	4.48 s	28.6 MB
Metric regeneration from calibration traces	72	Mac mini M4, 16 GB, macOS 26.4.1, Python 3.12.6	0.54 s	25.2 MB
Full release validation	396	Mac mini M4, 16 GB, macOS 26.4.1, Python 3.12.6	12.45 s	10.8 MB

E Asset Licenses

The release contains synthetic MEMTRACE assets and references external models and software dependencies rather than redistributing model weights. Tables 12 and 13 list the assets that affect the reported runs. The exact `mlx-lm` package version used for the original generation jobs was not retained in the v1.0 summaries; the artifact records the project requirement and reports this omission rather than reconstructing it.

Table 12: External models and backends referenced by the v1.0 artifact. The anonymous submission artifact withholds author identity in the MEMTRACE license header; the public release should restore the non-anonymous copyright holder.

Asset	Version/revision	Role	License
mlx-community/Qwen2.5-7B-Instruct-4bit	cached HF rev. c26a38f6; converted from Qwen with mlx-lm 0.18.1	actor model	Apache-2.0
Qwen tokenizer bundled with actor	cached HF rev. c26a38f6	tokenization	Apache-2.0
mlx-lm	requirement ≥ 0.21 ; local verification 0.31.3; generation package version not retained	generation back-end	MIT
BAAI/bge-small-en-v1.5	cached HF rev. 5c38ec7c	dense retrieval	MIT

Table 13: Python dependencies and MEMTRACE assets in the anonymous review artifact.

Asset	Version/revision	Role	License/status
sentence-transformers	requirement ≥ 3.0 ; local verification 5.4.1	embedding runtime	Apache-2.0 dependency
numpy	requirement ≥ 1.26 ; local verification 2.4.4	dense-index arrays	BSD-3-Clause family dependency
pydantic	requirement ≥ 2.0 ; local verification 2.13.3	schemas	MIT dependency
pytest	requirement ≥ 8.0 ; local verification 9.0.3	tests	MIT dependency
MEMTRACE synthetic corpus	v1.0	benchmark asset	MIT included
MEMTRACE traces	v1.0 main 324 plus 72 calibration	benchmark asset	MIT included

F Attempted Runs and Exclusions

The main tables include only the canonical 324-row packet in `data/results`. Table 14 lists the other local run packets considered during preparation and why they are not used as additional evidence. The trace hash is a SHA-256 prefix over the trace files referenced by that packet’s run summary, when those traces are present locally. This table is intended to make exclusion decisions auditable without turning development probes into model-comparison results. Table 14 is an audit log, not a model-comparison table; excluded rows are development probes and cannot support empirical claims.

Post-audit rerun policy. We froze the scorer source identified by scorer code v1 (git base 446d2d90a8a4; `memtrace/eval/scoring.py` SHA-256 prefix 43bb0da1666a; `memtrace/eval/metrics.py` SHA-256 prefix bd1c5da7a28f), regenerated all included rows from the same episode specifications, and excluded development runs that failed declared instrumentation gates. Actor codes are Q7=mlx-community/Qwen2.5-7B-Instruct-4bit, Q3=mlx-community/Qwen2.5-3B-Instruct-4bit, L31=mlx-community/Meta-Llama-3.1-8B-Instruct-4bit, and L32=mlx-community/Llama-3.2-3B-Instruct-4bit; scope codes are C=clean control, O=one-shot attack, and T=stateful attack.

No excluded packet is used to support model-comparison, prevalence, or defense-superiority claims.

Table 14: Included and excluded local run packets. Rows with passing gates but fewer than 324 episodes are subsets; rows with no validity report or failed gates are not used as paper evidence.

Run packet and hash	Rows	Actor	Scope	Scorer	Inclusion decision
data/results hash aff1934e11	324	Q7	S0-S2; C/O/T	v1	included main evidence packet
data/pilot/budget_s0_clean_chatfix hash 39acc48e8d	1	Q7	S0; C	v1	excluded: failed gates
data/pilot/budget_s0_oneshot_chatfix hash fe2a9ad948	1	Q7	S0; O	v1	excluded: failed gates
data/pilot/calendar_location_s0_ stateful_one hash 129ba60a05	1	Q7	S0; T	v1	excluded: failed gates
data/pilot/calendar_s1_stateful_chatfix hash 0a77a9c89d	1	Q7	S1; T	v1	excluded: failed gates
data/pilot/focused_qwen_v1 hash 197040ae95	81	Q7	S0-S2; C/O/T	v1	excluded: failed gates
data/pilot/focused_qwen_v2 hash f2b82cc82c	54	Q7	S0-S2; C/O/T	v1	excluded: incomplete passing subset
data/pilot/latest hash ce3f846149	54	Q7	S0-S2; C/O/T	v1	excluded: incomplete passing subset
data/pilot/llama32_3b_neurips_probe_v1 hash 84add58618	45	L32	S0-S2; C/O/T	v1	excluded: failed gates
data/pilot/llama_calendar_attendee_full_ v1 hash 357e1815ff	4	L31	S0; C/O/T	v1	excluded: no validity report
data/pilot/llama_calendar_attendee_v3 hash e8fbbdb08e	18	L31	S1/S2; C/O/T	v1	excluded: failed gates
data/pilot/llama_neurips_probe_v1 hash 7572a3da47	6	L31	S0; C/O/T	v1	excluded: failed gates
data/pilot/llama_rescue_toolfix_v1 hash 21e6c90568	3	L31	S2; C/O/T	v1	excluded: failed gates
data/pilot/meeting_time_s0_clean hash 6db65c742f	1	Q7	S0; C	v1	excluded: failed gates
data/pilot/qwen25_3b_neurips_probe_v1 hash 97714dc685	45	Q3	S0-S2; C/O/T	v1	excluded: failed gates
data/pilot/qwen25_corrected_affected_ slice hash 95f91608db	81	Q7	S0-S2; C/O/T	v1	excluded: failed gates
data/pilot/qwen25_corrected_merged hash aff1934e11	324	Q7	S0-S2; C/O/T	v1	excluded: superseded 324-row candi- date
data/pilot/qwen_calendar_attendee_ confirm_v1 hash ba463a0767	4	Q7	S0; C/O/T	v1	excluded: no validity report
data/pilot/qwen_expense_toolfix_v1 hash aa6df82234	9	Q7	S2; C/O/T	v1	excluded: failed gates
data/pilot/qwen_gate_repair_v1_approval_ limit hash 254aaaae982	27	Q7	S0-S2; C/O/T	v1	excluded: failed gates
data/pilot/qwen_gate_repair_v1_calendar_ attendee_s1 hash 53366ae536	9	Q7	S1; C/O/T	v1	excluded: failed gates
data/pilot/qwen_gate_repair_v1_calendar_ location_s0 hash 7a37f2bb61	9	Q7	S0; C/O/T	v1	excluded: failed gates
data/pilot/qwen_gate_repair_v1_meeting_ time hash 1f1d335102	27	Q7	S0-S2; C/O/T	v1	excluded: failed gates
data/results_qwen_repaired_v1 hash 6e1efba10e	324	Q7	S0-S2; C/O/T	v1	excluded: superseded 324-row candi- date
data/results_qwen_toolfix_v1 hash ad891adf60	324	Q7	S0-S2; C/O/T	v1	excluded: superseded 324-row candi- date
data/results_qwen_v3_full hash d5ad3f928d	94	Q7	S0; C/O/T	v1	excluded: no validity report
data/calibration/s1_oracle_retrieved_ memory hash 6d8542da44	72	Q7	S1-ORACLE; T	v1	included calibration-only sensitivity packet; excluded from main rates

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction state the benchmark, protocol, and empirical scope; the results and limitations sections explicitly say that included runs show no delayed exploitation and support only the admission-surface claim.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The limitations section states that the v1.0 audited pilot result set uses one pilot-valid actor/backend pair, a compact enterprise-assistant domain, deterministic tools, and fixed retrieval.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include theoretical results or formal proofs.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.

- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The protocol section and release describe fixed retrieval, prompts, deterministic tool stubs, metrics, traces, and validity gates needed to reproduce the included results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The release bundle contains the synthetic corpus, allowlist, episodes, gold labels, prompts, traces, aggregate metrics, figures, audit packet, and harness code.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.

- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The benchmark and protocol sections specify systems, task families, payload types, horizons, actor/backend setting, decoding, retrieval order, tool outputs, and validity gates.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The runs are deterministic fixed-protocol evaluations; the paper reports exact episode counts and rates with Wilson 95% finite-episode intervals, explicitly described as finite-episode uncertainty rather than stochastic decoding variability.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.

- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Appendix Tables 10 and 11 report episode counts, actor/backend metadata, local reproducibility hardware, measured trace-regeneration and readiness-check runtimes, and explicitly state which original generation telemetry was not retained.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: [Yes]

Justification: The work uses synthetic defensive benchmark tasks, no human-subject data, and reports dual-use risks and mitigations in the ethics and broader-impacts section.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The ethics and broader-impacts section discusses defensive benefits, possible misuse of attack payloads, and release mitigations.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [\[Yes\]](#)

Justification: The release keeps payloads synthetic and domain-specific, includes provenance-aware filtering and deterministic policy checks, and provides an audit packet for review.

Guidelines:

- The answer [\[N/A\]](#) means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: Appendix Tables 12 and 13 and the release file `THIRD_PARTY_ASSETS.md` list the actor model, tokenizer, inference backend, embedding model, Python dependencies, MEMTRACE corpus, and MEMTRACE traces with licenses and sources.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.

- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: MEMTRACE introduces new synthetic benchmark assets and includes a dataset card, generated traces, labels, metrics, and reproducibility harness in the release bundle.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[N/A\]](#)

Justification: The benchmark does not include crowdsourcing experiments or research with human subjects.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [\[N/A\]](#)

Justification: The benchmark uses synthetic tasks and generated traces, not human-subject data, so IRB approval is not applicable.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.

- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [\[Yes\]](#)

Justification: LLMs are the evaluated agents in the experiments; drafting and code-assistance tools may also be used, with authors responsible for checking claims, code, and final text.

Guidelines:

- The answer [\[N/A\]](#) means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.